Why This Is the Right Solution

A detailed guide to the Budget Traveller with Coupons algorithm

Short answer: The route is constrained by both the number of flights and the number of coupons spent. Expanded-state Dijkstra keeps those resources in the search state, explores every legal coupon choice, and returns the cheapest feasible journey.

1. The problem in plain terms

Cities are vertices and one-way flights are directed edges. Each edge has a nonnegative integer price. A route from city 1 to city N may use at most K intermediate cities, which means at most K + 1 flights. Up to C different flights may be discounted; each discounted fare is floor(price / 2). Coupons can remain unused.

The objective is not simply “find the cheapest path.” It is “find the cheapest path among paths that satisfy both limits.” A cheapest unrestricted route may use too many flights. A coupon spent early may prevent a larger saving later.

2. Why ordinary shortest path is insufficient

A normal shortest-path algorithm stores one best cost per city. That merges arrivals that are not equivalent. Reaching city 4 for cost 200 after one flight leaves more flight allowance than reaching city 4 for cost 200 after three flights. Reaching it with a coupon available differs from reaching it after spending the last coupon.

State detail	What future choice it controls
Current city	Which outgoing flights can be taken.
Flights already used	Whether another flight exceeds K + 1.
Coupons already used	Whether a discounted transition remains legal.

If the search merges states only because they are at the same city, it can discard the only feasible continuation or the optimal coupon allocation. The expanded state preserves exactly the information that affects future choices.

3. Expanded-state Dijkstra

Represent a search state as **(city, flights_used, coupons_used)**. Begin at (1, 0, 0) with cost zero. For each outgoing flight u -> v with price p, consider up to two transitions:

Action	New state	Added cost	Condition
No coupon	(v, flights+1, coupons)	p	Flight allowance remains
Use coupon	(v, flights+1, coupons+1)	p // 2	coupons_used < C

A min-priority queue processes the currently cheapest state first. All transition costs are nonnegative. Dijkstra's guarantee therefore applies: once a state is popped at its best cost, no later route can reach that exact state more cheaply. Stop when a destination state is popped; it is the lowest-cost valid destination state, no matter how many coupons it used.

Algorithm outline

```
push (0, (1, 0, 0)) into min-heap
best[(1, 0, 0)] = 0
while heap is not empty:
    cost, (city, flights, coupons) = pop minimum
    if cost is stale: continue
    if city == N: reconstruct and return
    if flights == K + 1: continue
    for each flight (city -> next_city, price):
        relax full-price state at cost + price
        if coupons < C:
            relax discounted state at cost + price // 2
return -1
```

A relaxation updates a state only when the new total is lower than the known total. Its predecessor record stores the precise flight and coupon flag; this recovers the route and handles parallel flights.

4. Why the result is globally optimal

Completeness: Every legal route prefix becomes a sequence of expanded states. Full-price and discounted choices are both represented, while the counters reject transitions beyond K + 1 flights or C coupons. Thus every allowed journey is searched and every searched journey obeys the rules.

Minimum cost: Since fares and half-fares are nonnegative, Dijkstra finalizes states in increasing true cost. The first destination state removed from the queue is therefore the least-cost valid journey. The algorithm does not guess where to use a coupon; it compares the consequences of each placement.

5. Example: save the coupon for later

Let K = 1 and C = 1. Flights: 1 -> 2 costs 4; 2 -> 4 costs 101; 1 -> 3 costs 60; 3 -> 4 costs 60. The route through city 2 costs 105 without a coupon. Discounting its first flight gives 2 + 101 = 103. Saving the coupon for the expensive second flight gives 4 + 50 = 54. The alternate route costs 120 without a coupon or 90 with one coupon. The optimum is 54.

Route	Coupon use	Total cost
1 -> 2 -> 4	None	4 + 101 = 105
1 -> 2 -> 4	First flight	2 + 101 = 103
1 -> 2 -> 4	Second flight	4 + 50 = 54
1 -> 3 -> 4	One 60 fare	60 + 30 = 90

A greedy “discount the first flight” rule returns 103 instead of 54. The state search keeps coupon use explicit and compares both branches.

6. Stop limit and edge cases

K counts intermediate cities, not flights. Let F = K + 1. K=0 permits only direct flights; K=1 permits up to two flights. A route with L flights uses max(0, L-1) intermediate stops when start and destination differ. The solver accepts arrival on flight F and does not expand states that have already used F flights.

Zero-price edges: valid in Dijkstra; best-cost updates prevent endless cycling. **Cycles:** flight count is bounded and only improved states are queued. **Parallel flights:** distinct edge records remain distinct. **Rounding:** price 101 becomes 50 and price 1 becomes 0. **Unused coupons:** full-price transitions are always available. **No route:** return -1. **N=1:** the empty trip costs 0.

7. Complexity and what “best case” means

This question can mean either “why is this the right algorithm?” or the formal best-case runtime. The algorithm is the right fit because it is correct with the two constraints and efficient for nonnegative prices. Its runtime is not constant; it depends on how much of the expanded state graph is explored.

Let F = K + 1. There are at most N × (F + 1) × (C + 1) states: each city, each flight count from 0 to F, and each coupon count from 0 to C. From each state, every outgoing original flight creates at most two transitions. Thus E' is at most about 2 × M × F × (C + 1).

Quantity	Upper bound / interpretation
Expanded states V'	N × (F + 1) × (C + 1)
Expanded transitions E'	About 2 × M × F × (C + 1)
Heap Dijkstra time	O((V' + E') log V')
Memory	Distances and predecessors for discovered states, plus original graph

Formal best case: the optimal destination state is popped after a small part of the reachable state graph has been visited. If a cheap direct flight is the lowest-cost queue candidate, the search may stop after checking the outgoing flights from city 1 and popping the destination. It must still inspect those outgoing flights, so work depends at least on the start-city degree. There is no universal O(1) guarantee.

Worst case: many combinations of city, flight count, and coupons are reachable before the destination is finalized. If no route exists, the search may exhaust every reachable state. Increasing K or C grows the state space. K is commonly small, which makes this practical.

8. Why not use a simpler alternative?

Alternative	Limitation for this problem
BFS	Optimizes flight count, not ticket cost.
Dijkstra by city only	Forgets used flights and coupons remaining.
Bellman-Ford	Negative fares are not present; resource counters are still needed.
Enumerate every route	The number of paths and coupon placements grows quickly.
Greedy coupon placement	May discount a cheap early flight instead of a costly later one.

Expanded-state Dijkstra combines the right cost ordering with the bookkeeping required by the constraints. It finds an optimum without enumerating every full journey.

9. Project use and verification

Run the GUI from the project folder after installing requirements:

```
cd budget-traveller
python3 -m pip install -r requirements.txt
python3 main.py
```

Run correctness tests with:

```
python3 -m unittest discover -s tests -v
```

The test suite covers direct routes, coupons, stop limits, unreachable destinations, cycles, parallel edges, zero and odd prices, N=1, and invalid inputs. Seeded random cases are compared with a separate layered dynamic-programming reference solver.

10. Takeaway

Store **city + flights used + coupons used** as the state. That makes every relevant future choice explicit. Because every transition cost is nonnegative, Dijkstra can finalize states from lowest to highest cost. Together, these facts make the solution correct, reconstructable, and efficient for the problem.